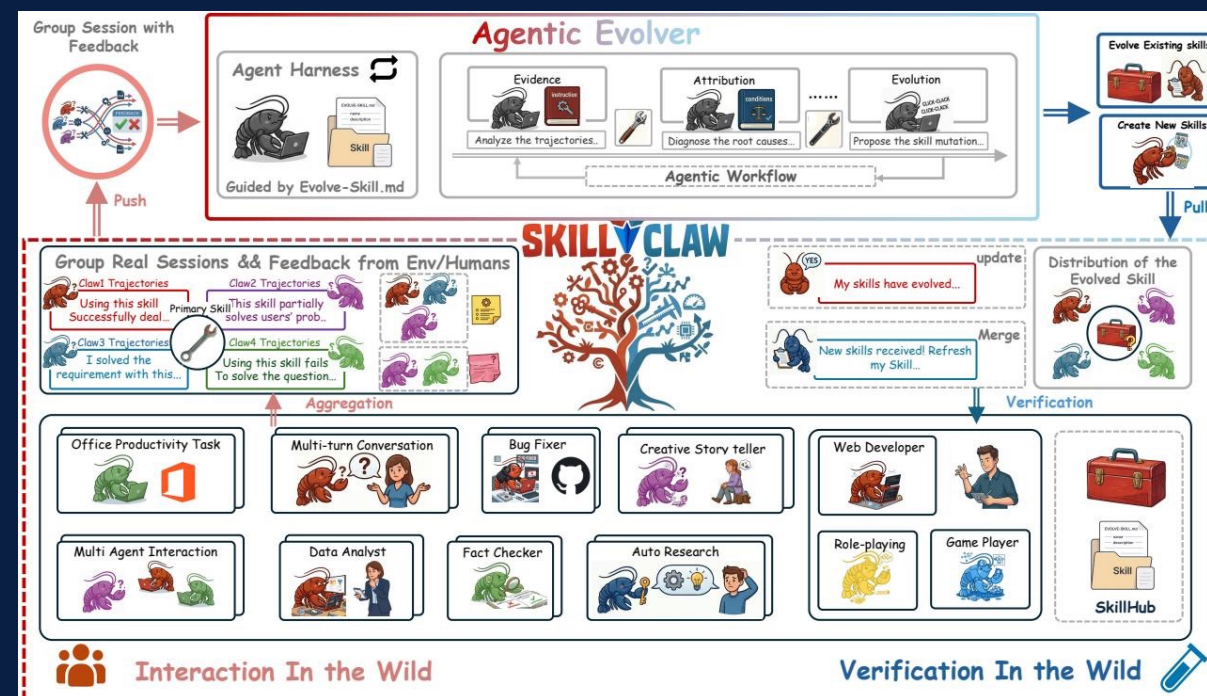


SkillClaw

Let Skills Evolve Collectively with Agentic Evolver

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

arxiv.org/abs/2604.08377



Agent Skills Remain Static After Deployment

Users Rediscover Solutions Independently

Static Skill Libraries

LLM agents rely on reusable skills, but these remain fixed after deployment. Users manually install skills from a centralized hub.

Session-Bound Knowledge

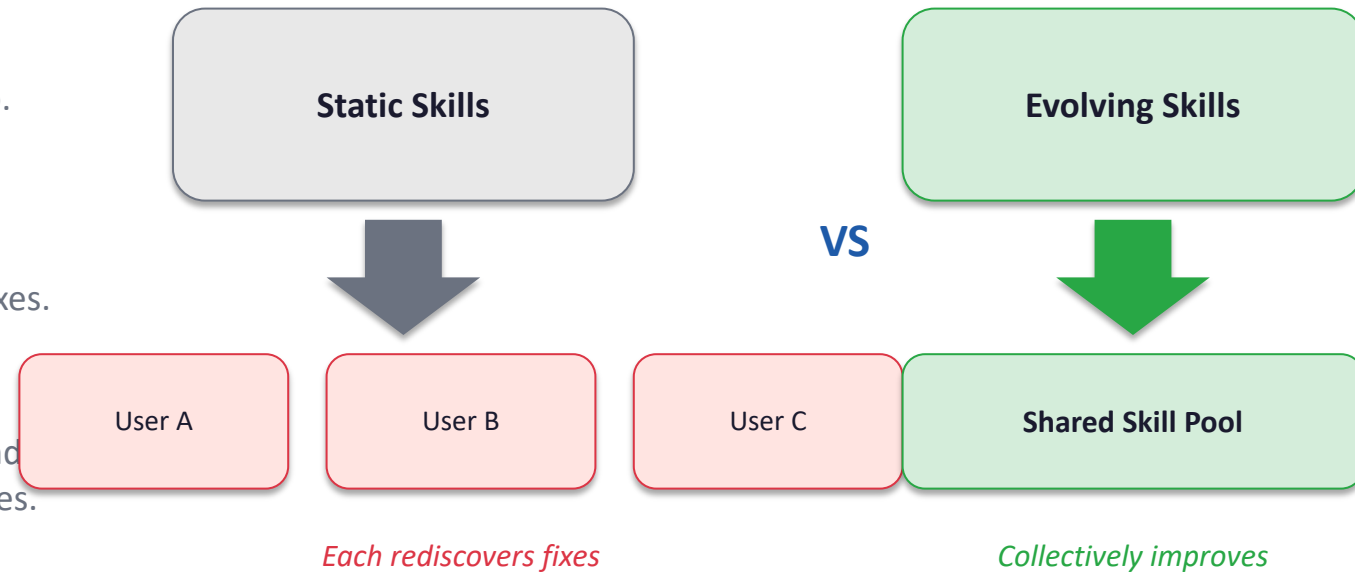
Solutions discovered during interaction rarely persist beyond individual sessions. Each user must independently rediscover fixes.

Repeated Failure Patterns

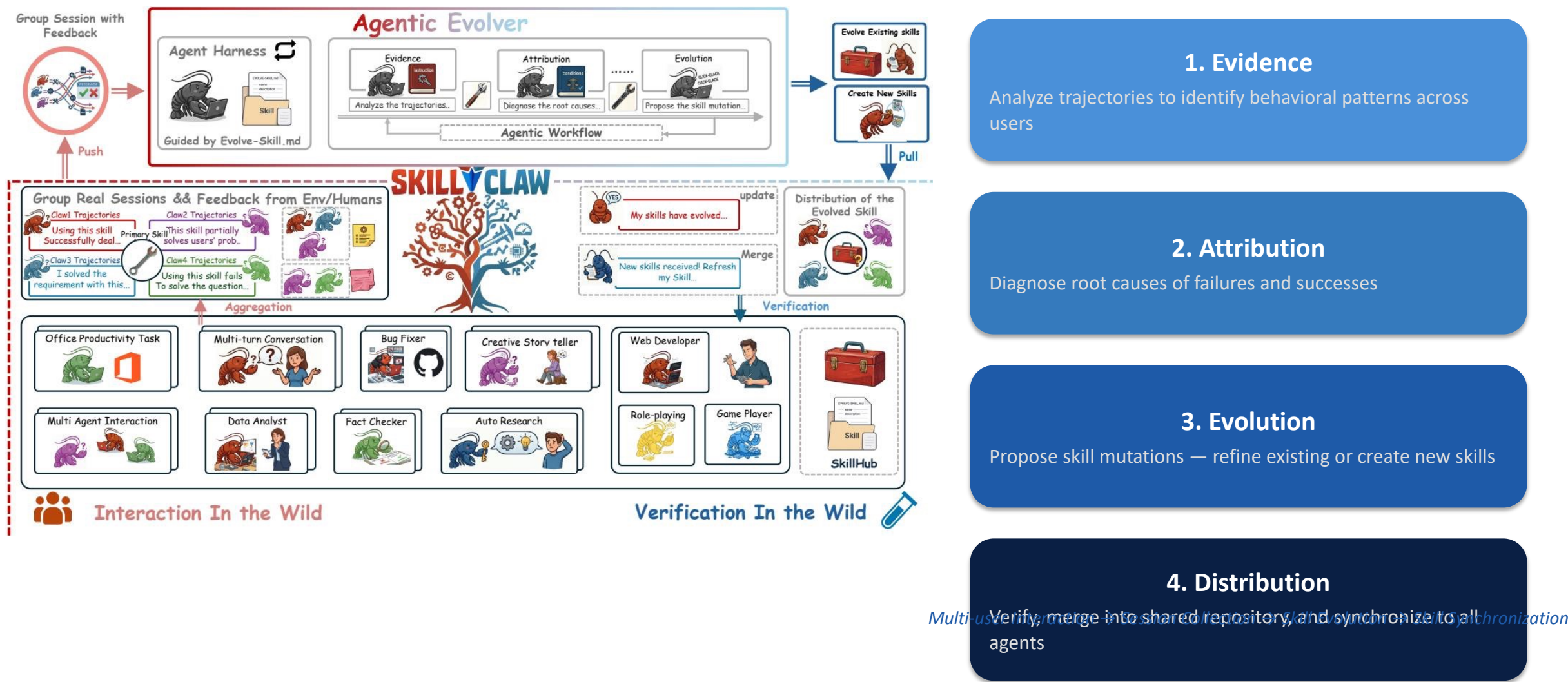
As similar tasks recur across users and time, the same failure and recovery patterns are observed — yet the system never improves.

Memory vs. Skill Gap

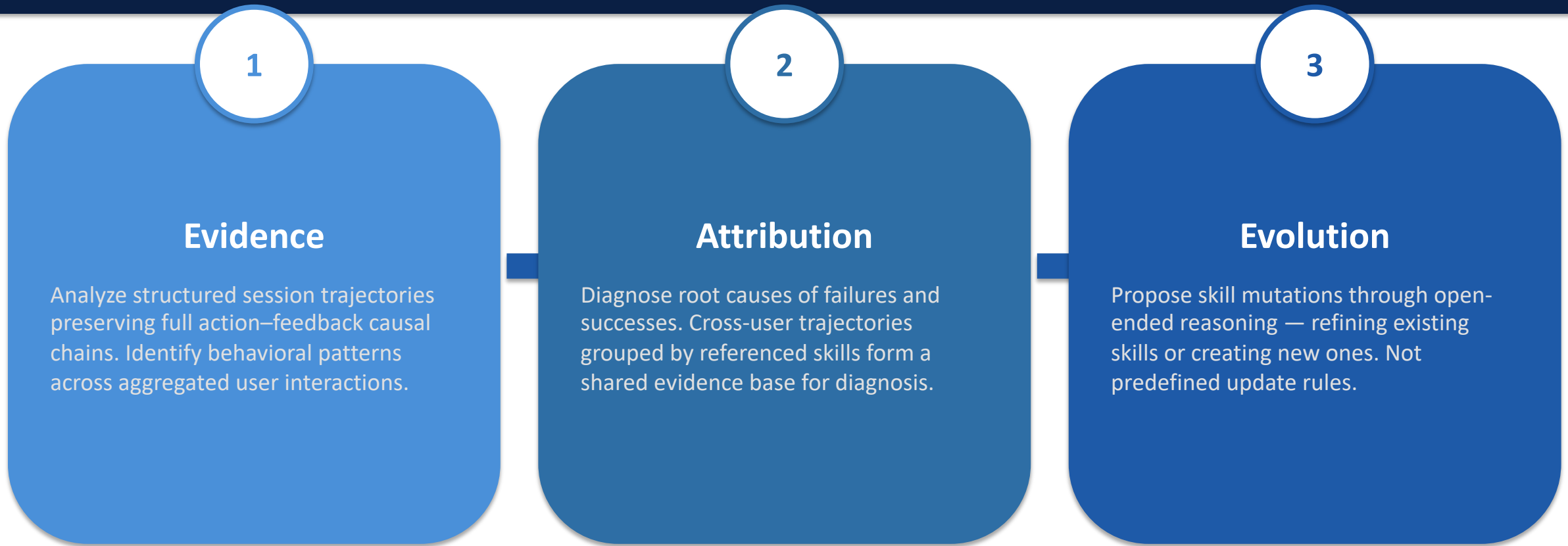
Memory-based methods store trajectories but are hard to generalize. Skill-based methods compress experience but treat the library as static.



Closed-Loop Pipeline: From Multi-User Interaction to Shared Skill Evolution



Agentic Evolver: Evidence → Attribution → Evolution in Three Stages



Key Insight: The evolution process is driven by an autonomous agent performing open-ended reasoning over interaction evidence, not by predefined update rules.

Cross-User Trajectory Aggregation Exposes Skill Behavioral Boundaries

Complementary Signals

Different users exercising the same skill under diverse contexts produce complementary views of that skill's behavioral boundary.

Success & Failure Modes

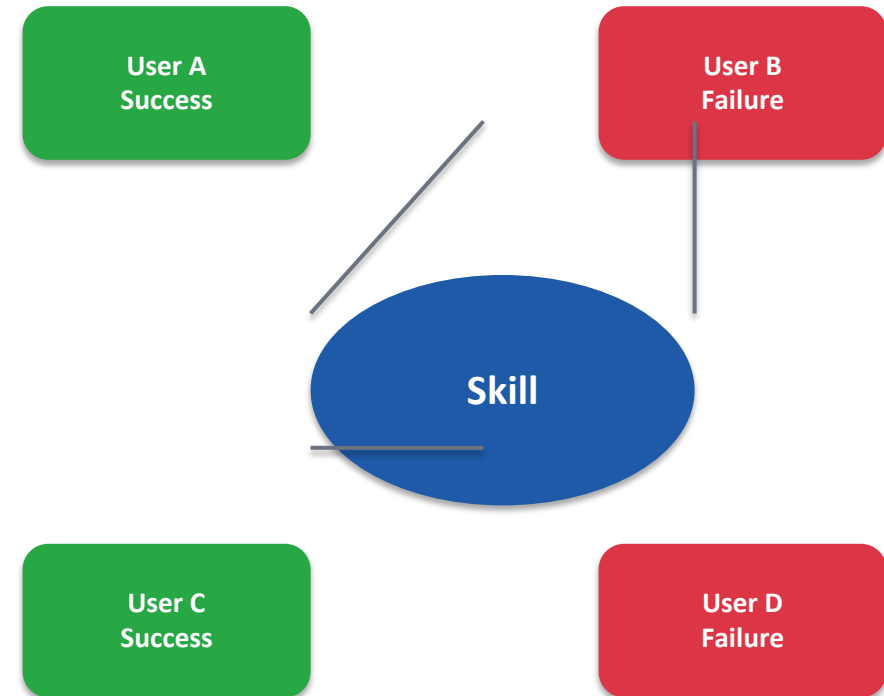
Cross-user aggregation reveals both conditions under which a skill works and those under which it breaks.

Single-User Limitation

A single user rarely generates enough signal to separate a generalizable improvement from an idiosyncratic fix.

Compatible Systems

SkillClaw works with OpenClaw, CoPaw, IronClaw, PicoClaw, ZeroClaw, NanoClaw, and NemoClaw.



Aggregated trajectories form shared evidence base

Case Study: Multimodal Creative Pipeline Skill

6 Days of Evolution Attempts — 1 of 6 Accepted

Day	Candidate Skill / Change	Validator	Next-Day Action
1	validate-tmp-workspace-inputs — Check /tmp_workspace inputs & environment setup	Accept	Upgrade to new best pool
2	multimodal-input-validation-and-setup — Multimodal input validation & output env init	Reject	Keep current best pool
3	multimodal-creative-task-pipeline — Multimodal creative pipeline (extract from PDF/video/image)	Reject	Keep current best pool
4	multimodal-creative-task-pipeline (impr.) — Added image classification, visual generation, structured validation	Reject	Keep current best pool
5	multimodal-creative-task-pipeline (impr.) + validate-required-input-files — Creative pipeline + per-file fail-fast validation	Reject	Keep current best pool
6	multimodal-creative-task-pipeline (cand.) — Extended PDF-to-poster / document-to-visual paths	Reject	Not admitted to next cycle

Insight: Conservative verification prevents regressions. Only 1 of 6 candidates was accepted, demonstrating rigorous quality control.

Case Study: Git Push Auth-Fallback Skill

Iterative Refinement Over 6 Days — 3 of 6 Accepted

Day	Candidate Skill / Change	Validator	Next-Day Action
1	git-push-with-auth-fallback — Safe fallback instead of blocking on push failure	Accept	Add to Safety best pool
2	git-push-with-auth-fallback — Unified patch/bundle filenames, reduced inconsistency	Accept	Keep updated best pool
3	git-push-with-auth-fallback + git-clone-to-directory — Auth-alternative paths & secrets audit; fixed subshell pitfalls	Accept	Keep current best pool
4	(none) — Same-pool retest; validator confirmed current pool as best	Accept	Continue same best pool
5	git-push-with-auth-fallback — Added 'push hang treated as auth failure'; no improvement	Reject	Keep current best pool
6	git-push-with-auth-fallback — Added identity config & filename consistency; no improvement	Reject	Not admitted to next cycle

Insight: Iterative refinement works when evidence supports it. The skill was successfully improved 3 times before reaching a plateau.

Key Takeaways: Three Properties That Distinguish SkillClaw

Collective Evolution

Knowledge from individual interactions contributes to a shared, continuously improving skill ecosystem. The system learns from all users simultaneously.

Fully Automatic

Skill evolution is driven by runtime interaction without manual curation or explicit user intervention. No human-in-the-loop required for skill updates.

Agentic Evolution Paradigm

Skill updates are produced through open-ended reasoning over evidence rather than predefined update rules. The evolver reasons autonomously about what to change and why.

Implications for Building Continuously Improving Agent Systems

- Static skill ecosystems force users to independently rediscover solutions — collective evolution breaks this cycle.
- Cross-user trajectory aggregation provides signals no single user can generate alone.
- Conservative verification (1/6 accepted in creative case) prevents regressions while allowing iterative refinement (3/6 in git case).
- Compatible with Claw-style systems; evaluated on WildClawBench with Qwen3-Max demonstrating substantial improvements.